

割当自動生成ロジック

KARAKIDA-PORTAL — assignment.js

1. 全体フロー (awGenerateAll)

「自動生成」ボタン押下時に実行される。表示中の月の全週に対し、週ごとに順番に割当を生成する。

処理ステップ

1. 対象抽出 — 表示中の月でフィルタした週一覧を取得
2. 仮履歴 (tempHistory) を作成 — Firestore の assignmentHistory をディープコピー
3. 週ごとにループ（日付順）：
 - 大会週 (conventionType あり) → スキップ
 - その週のプログラム項目から割当コード一覧を取得
 - awRunGeneration() でコア生成を実行
 - 生成結果を仮履歴に書き戻し（次週の生成に反映）
4. UI のセレクトボックスに結果を反映

仮履歴の積み上げ（週間ローテーション）

Week1 生成後、tempHistory に割当結果を反映する:

```
tempHistory[名前][base].lastDate = Week1 の meetDate  
tempHistory[名前][base].count++
```

Week2 の生成時には更新済みの tempHistory を参照するため、

Week1 で割当てた人は daysSince=7 となりスコアが下がり、別の人が優先される。

2. コア生成ロジック (awRunGeneration)

入力パラメータ

- allCodes — その週の全割当コード (例: A, B, C, D, E, F, H, I, J, K ...)
- members — 生徒リスト (eligibleCodes, gender, familyGroup, position)
- history — 割当履歴 {名前: {コード: {lastDate, count}}}
- meetDate — その週の集会日 (木曜日)

Step 1: コードを候補者数の少ない順にソート

各コードについて、そのコードの `eligibleCodes` を持つメンバー数をカウントし、少ない順にソートする。候補者が限られる役割を先に割当てすることで、割当不能を防止する。

```
sortedCodes = allCodes.sort(候補者が少ない順)
```

例: 書朝読(候補3名) → 談話担当(候補8名) → 司会(候補12名)

Step 2: 候補者フィルタリング

各コードに対し、以下の条件で候補者を絞り込む:

- `eligibleCodes` にそのコードの `base` を持っていない → 除外
- 同じ週内で既に別の役割に割当済み (`assignedPersons`) → 除外
- 野外奉仕コード (F~Q) かつ `position` が「長老」 → 除外

Step 3: ペア制約チェック (相手コードの場合)

担当・相手のペア関係: $H \leftrightarrow I$, $J \leftrightarrow K$, $L \leftrightarrow M$, $N \leftrightarrow O$

相手コード (I, K, M, O) を処理する際:

- 先に割当済みの担当者の性別と候補者の性別を比較
- 異性ペアの場合 → 同じ `familyGroup` でなければ除外
- 同性 or 性別未設定 → 制約なし (OK)

3. スコア計算

基本式

```
score = daysSince × 10 - count
```

```
daysSince = meetDate - lastDate (日数)
```

※未割当なら9999

```
count = 過去の割当回数
```

→ スコアが高い = 長期間割当がない & 回数が少ない → 優先的に選ばれる

→ 最高スコアの候補者がその役割に割当てられる

野外奉仕コード (F~Q) の場合: 横断スコア

野外奉仕コードの場合、特定コードだけでなく全野外奉仕コード (F~Q) を横断して最新の割当日と累計回数を算出する。

全野外奉仕コード(F~Q)を横断して:

```
latestDate = 最も最近の割当日
```

```
totalCount = 全コードの割当回数合計
```

```
daysSince = meetDate - latestDate
```

```
score = daysSince × 10 - totalCount
```

理由: 野外奉仕のコードは H=会話を始める担当、J=再び話し合う担当 のように項目ごとに異なるが、同じ生徒が毎週別の項目に出るのは不自然。全コード横断で見ること、先週どの項目に出た人でもスコアが下がり、別の人が優先される。

その他のコード (司会・祈り・講演等) の場合

そのコード固有の履歴 (base 単位) で lastDate と count を取得してスコアを算出する。

例: 司会者 (A) の履歴は A のみ参照。宝石を探し出す (D) の履歴は D のみ参照。

Step 5: 割当決定

```
candidates.sort(スコア降順)
```

```
result[code] = candidates[0] // 最高スコアの人
```

```
assignedPersons.add(選ばれた人) // 同一週内の重複防止
```

4. コード体系

割当コード一覧

コード	役割	区分
A	司会者	開会
B	祈り（開会）	開会
C	神の言葉の宝 講演1	講演
D	宝石を探し出す	講演
E	聖書朗読	講演
F, G	聖書朗読・朗読者	野外奉仕
H / I	会話を始める 担当/相手	野外奉仕ペア
J / K	再び話し合う 担当/相手	野外奉仕ペア
L / M	3つ目の生徒割当 担当/相手	野外奉仕ペア
N / O	4つ目の生徒割当 担当/相手	野外奉仕ペア
P～	その他生徒	野外奉仕
R～	クリスチャンとして生活する	講演

ペア関係

ペア	担当コード	相手コード	備考
1	H	I	会話を始める
2	J	K	再び話し合う
3	L	M	3つ目の生徒割当
4	N	O	4つ目の生徒割当

5. 制約まとめ

制約	内容
同一週内重複なし	assignedPersons で管理。1人1週1役割のみ
長老は生徒割当除外	position=長老 かつ野外奉仕コード(F～Q) → 除外
異性ペアは家族のみ	担当と相手が異性の場合、familyGroup 一致が必要
野外奉仕は横断スコア	F～Q 全体の最新割当日・累計回数で判定
大会週スキップ	conventionType あり → 生成しない
週間ローテーション	tempHistory で仮履歴を積み上げて連続週を防止

6. 処理フローチャート

